Credit Card Fraud Detection Project

Model Performance Evaluation and Statistical Analysis

Student:

Selime Kör

In [1]:
```
# 1. GEREKLİ KÜTÜPHANELER
# ==========================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import norm

from sklearn.model_selection import (
    train_test_split,
    cross_val_score
)

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
```

Veri Seti Yükleme

Bu bölümde kredi kartı dolandırıcılığı veri seti yüklenmiş ve analiz için hazırlanmıştır.

In [2]:
```
# 2. VERİ SETİNİ YÜKLEME
# ==========================================================

df = pd.read_csv("creditcard.csv")

print("İlk 5 Satır:")
print(df.head())
```

```
print("\nVeri Boyutu:")
print(df.shape)
```

```
İlk 5 Satır:
   Time        V1        V2        V3        V4        V5        V6        V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388  0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499  0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203  0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921  0.592941

         V8        V9  ...       V21       V22       V23       V24       V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928  0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846  0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575  0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]

Veri Boyutu:
(284807, 31)
```

In [3]:
```
# 3. VERİYİ KÜÇÜLTME (HIZ İÇİN)
# ==========================================================

# Büyük veri nedeniyle sistemi yormamak için
df_small = df.sample(50000, random_state=42)
```

Veri Keşfi ve Görselleştirme (EDA)

Bu bölümde veri setindeki sınıf dağılımı incelenmiş ve veri görselleştirme teknikleri kullanılmıştır.

Veri setinin dengesiz olduğu gözlemlenmiştir.

In [4]:
```
# 4. VERİ KEŞFİ (EDA)
# ==========================================================

print("\nSınıf Dağılımı:")
print(df_small['Class'].value_counts())

print("\nYüzdelik Dağılım:")
print(df_small['Class'].value_counts(normalize=True) * 100)
```

```
Sınıf Dağılımı:
Class
0    49917
1       83
Name: count, dtype: int64

Yüzdelik Dağılım:
Class
0    99.834
1     0.166
Name: proportion, dtype: float64
```

In [5]:
```
# 5. GÖRSELLEŞTİRME
# ========================================================

plt.figure(figsize=(6,4))

sns.countplot(x='Class', data=df_small)

plt.title("Fraud vs Normal Transactions")

plt.xlabel("Class")

plt.ylabel("Count")

plt.show()
```

In [6]:
```
# 6. ÖZELLİKLER VE HEDEF DEĞİŞKEN
# ========================================================

X = df_small.drop('Class', axis=1)

y = df_small['Class']
```

In [7]:
```
# 6. ÖZELLİKLER VE HEDEF DEĞİŞKEN
# ========================================================
```

```
X = df_small.drop('Class', axis=1)

y = df_small['Class']
```

In [8]:
```
# TRAIN TEST SPLIT

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [9]:
```
# 8. FEATURE SCALING
# ==========================================================

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Makine Öğrenmesi Modelleri

Bu bölümde kredi kartı dolandırıcılığı tespiti için farklı makine öğrenmesi modelleri eğitilmiş ve test edilmiştir.

Kullanılan modeller:

- Logistic Regression
- Decision Tree
- Random Forest

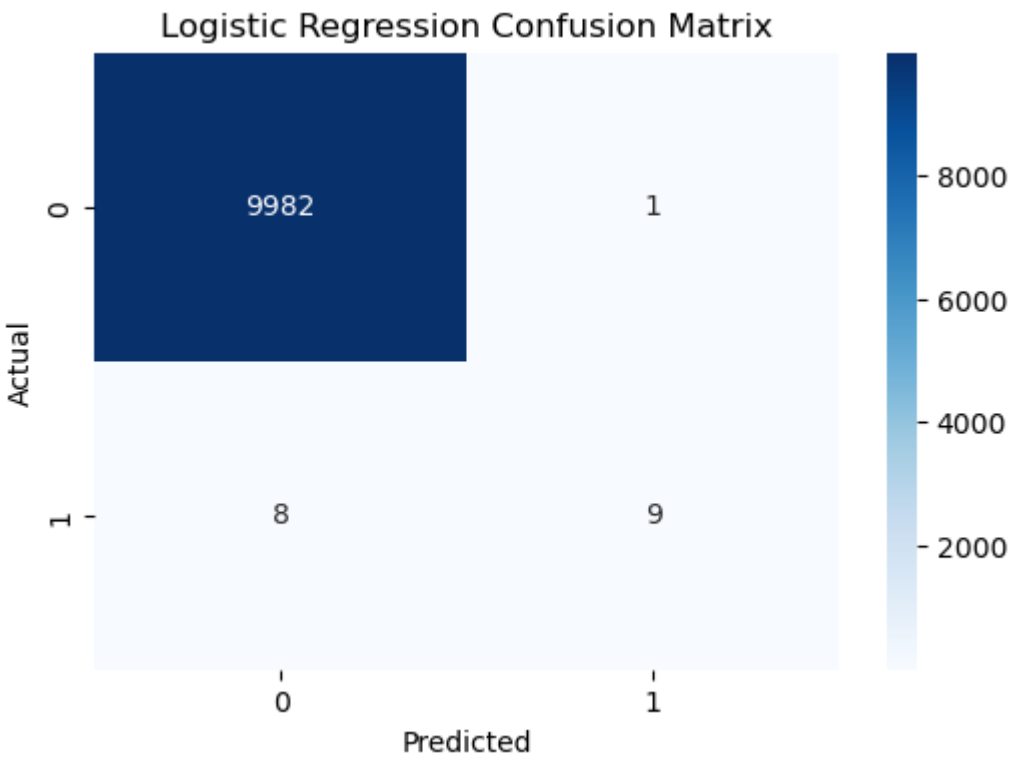
In [10]:
```
# 9. LOGISTIC REGRESSION
# ==========================================================

print("\n========== LOGISTIC REGRESSION ==========")

lr_model = LogisticRegression(max_iter=2000)

lr_model.fit(X_train_scaled, y_train)

lr_pred = lr_model.predict(X_test_scaled)

lr_accuracy = accuracy_score(y_test, lr_pred)

lr_precision = precision_score(y_test, lr_pred)

lr_recall = recall_score(y_test, lr_pred)

print("Accuracy:", lr_accuracy)

print("Precision:", lr_precision)
```

```
print("Recall:", lr_recall)
```

```
========== LOGISTIC REGRESSION ==========
Accuracy: 0.9991
Precision: 0.9
Recall: 0.5294117647058824
```

In [11]:
```
# 10. CONFUSION MATRIX
# ==========================================================

lr_cm = confusion_matrix(y_test, lr_pred)

print("\nConfusion Matrix:")
print(lr_cm)

plt.figure(figsize=(6,4))

sns.heatmap(lr_cm, annot=True, fmt='d', cmap='Blues')

plt.title("Logistic Regression Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

```
Confusion Matrix:
[[9982    1]
 [   8    9]]
```



In [12]:
```
# 11. CLASSIFICATION REPORT
# ==========================================================

print("\nClassification Report:")
print(classification_report(y_test, lr_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      9983
           1       0.90      0.53      0.67        17

    accuracy                           1.00     10000
   macro avg       0.95      0.76      0.83     10000
weighted avg       1.00      1.00      1.00     10000
```

In [13]:

```python
# 12. DECISION TREE
# ==========================================================

print("\n========== DECISION TREE ==========")

dt_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train_scaled, y_train)

dt_pred = dt_model.predict(X_test_scaled)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
========== DECISION TREE ==========
Decision Tree Accuracy: 0.9993
```

In [14]:

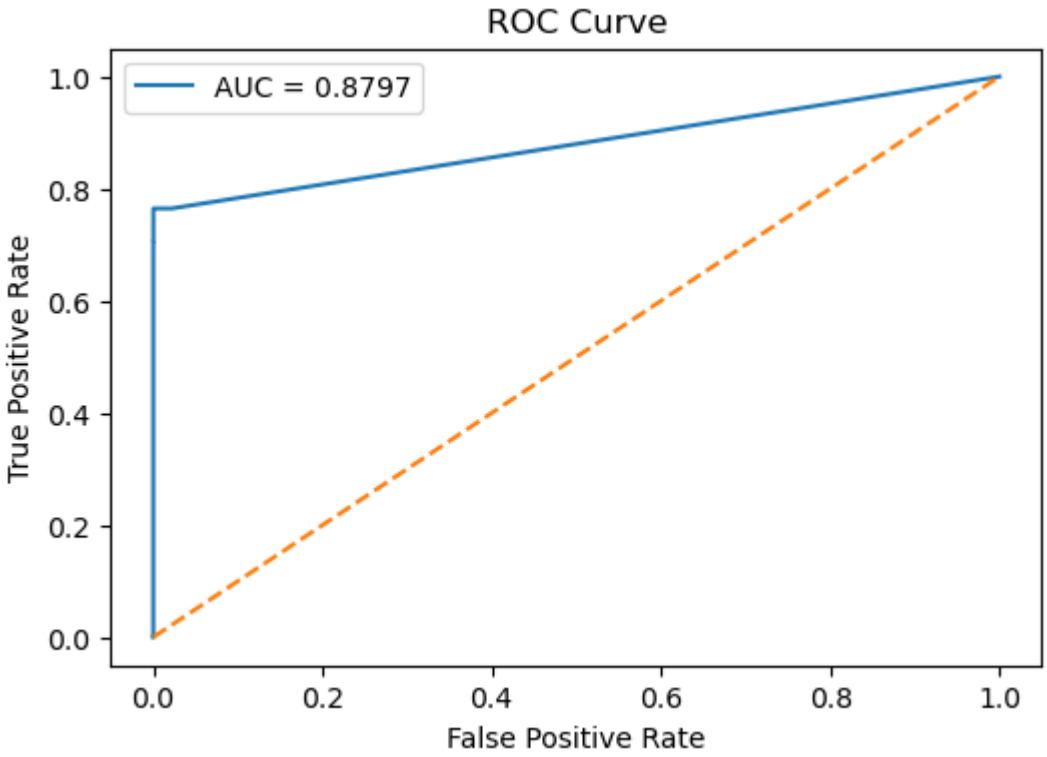
```python
# 13. RANDOM FOREST
# ==========================================================

print("\n========== RANDOM FOREST ==========")

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train_scaled, y_train)

rf_pred = rf_model.predict(X_test_scaled)

rf_accuracy = accuracy_score(y_test, rf_pred)

rf_precision = precision_score(y_test, rf_pred)

rf_recall = recall_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_accuracy)

print("Random Forest Precision:", rf_precision)

print("Random Forest Recall:", rf_recall)
```

```
========== RANDOM FOREST ==========
Random Forest Accuracy: 0.9993
Random Forest Precision: 0.9166666666666666
Random Forest Recall: 0.6470588235294118
```

In [15]:

```python
# 14. ROC CURVE & AUC
# ==========================================================

rf_probs = rf_model.predict_proba(X_test_scaled)[:,1]

fpr, tpr, thresholds = roc_curve(y_test, rf_probs)

roc_auc = auc(fpr, tpr)

print("\nAUC Score:", roc_auc)

plt.figure(figsize=(6,4))

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}")

plt.plot([0,1], [0,1], linestyle='--')

plt.title("ROC Curve")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.legend()

plt.show()
```

```
AUC Score: 0.8797131594298543
```



İstatistiksel Analiz

Bu bölümde model performansı istatistiksel yöntemler kullanılarak değerlendirilmiştir.

Kullanılan yöntemler:

- Confidence Interval
- Hypothesis Testing
- Cross Validation
- Central Limit Theorem

In [16]:
```
# 15. %95 GÜVEN ARALIĞI
# ==========================================================

print("\n========== CONFIDENCE INTERVAL ==========")

n = len(y_test)

p_hat = rf_accuracy

margin_error = 1.96 * np.sqrt((p_hat * (1 - p_hat)) / n)

lower = p_hat - margin_error

upper = p_hat + margin_error

print("Accuracy:", p_hat)

print("95% Confidence Interval:")

print(lower, upper)
```

```
========== CONFIDENCE INTERVAL ==========
Accuracy: 0.9993
95% Confidence Interval:
0.9987816142733446 0.9998183857266554
```

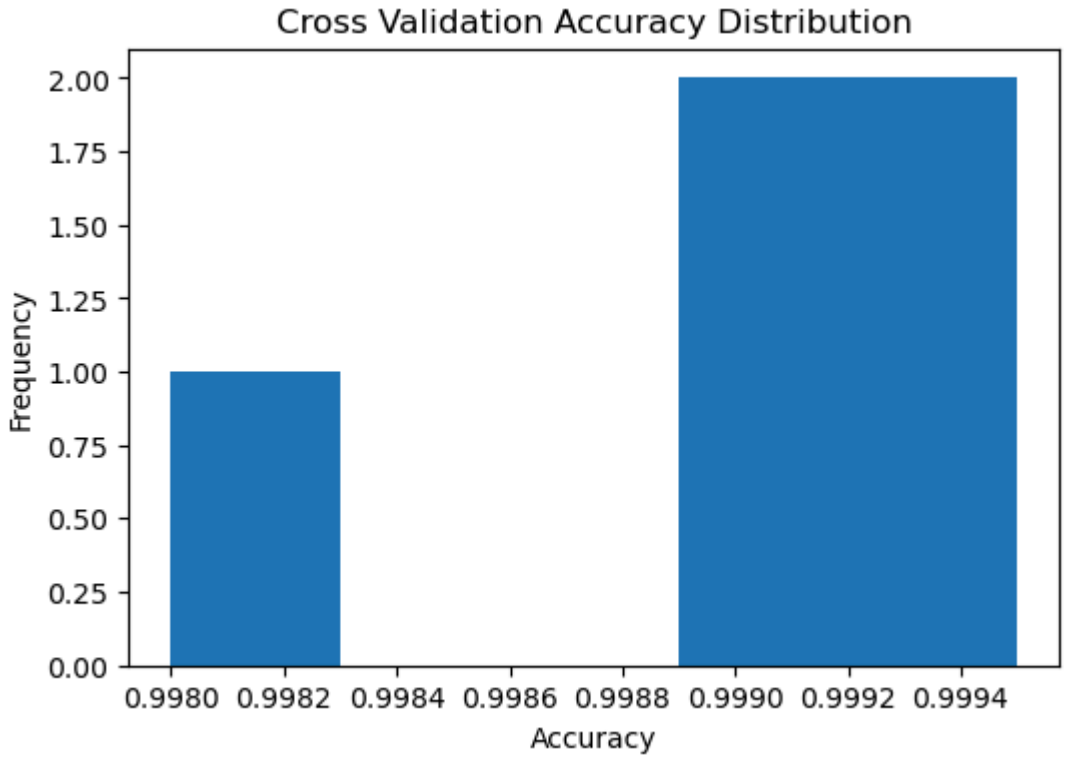
In [17]:
```
# 16. HİPOTEZ TESTİ
# ==========================================================

print("\n========== HYPOTHESIS TEST ==========")

# H0: Model accuracy <= 0.5
# H1: Model accuracy > 0.5

p0 = 0.5

z_score = (p_hat - p0) / np.sqrt((p0 * (1 - p0)) / n)

p_value = 1 - norm.cdf(z_score)

print("Z-score:", z_score)

print("P-value:", p_value)

if p_value < 0.05:
    print("H0 reddedilir.")
    print("Model istatistiksel olarak anlamlıdır.")
else:
    print("H0 reddedilemez.")
```

```
========== HYPOTHESIS TEST ==========
Z-score: 99.85999999999999
P-value: 0.0
H0 reddedilir.
Model istatistiksel olarak anlamlıdır.
```

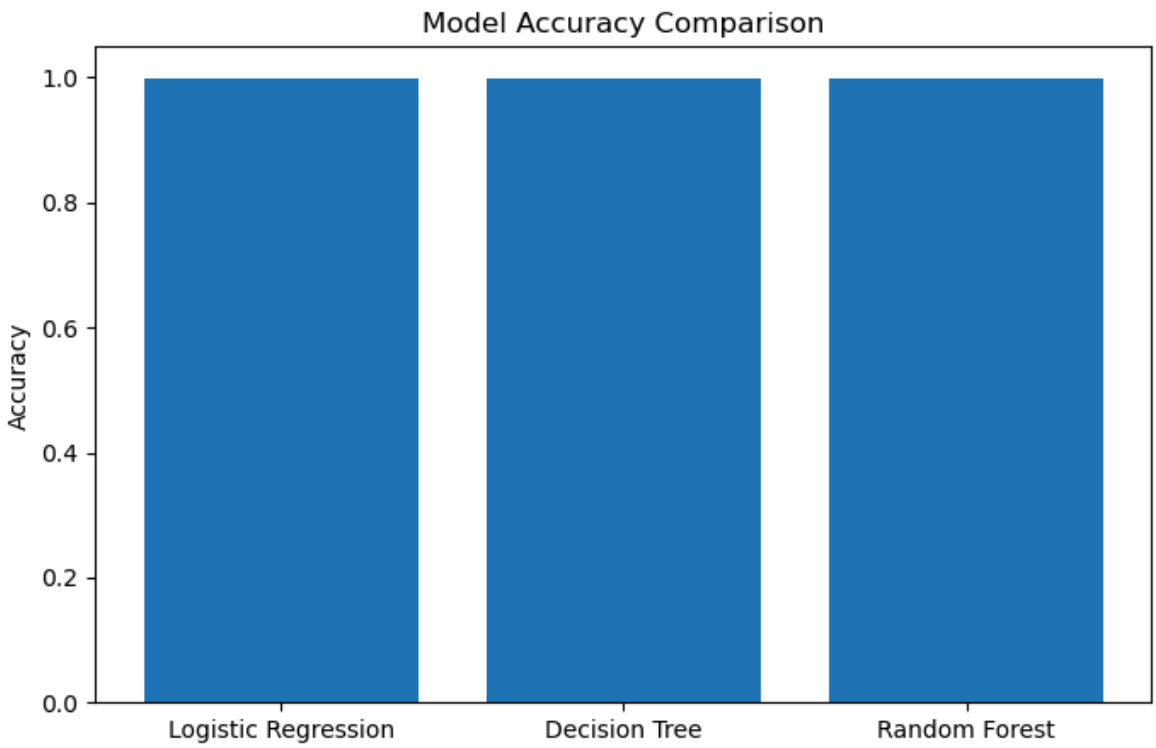
In [18]:
```python
# 17. 5-FOLD CROSS VALIDATION
# ==========================================================

print("\n========== CROSS VALIDATION ==========")

cv_scores = cross_val_score(
    rf_model,
    X_test_scaled,
    y_test,
    cv=5,
    scoring='accuracy'
)

print("CV Scores:")
print(cv_scores)

print("\nMean Accuracy:")
print(cv_scores.mean())

print("\nStandard Deviation:")
print(cv_scores.std())
```

```
========== CROSS VALIDATION ==========
CV Scores:
[0.999  0.9995 0.998  0.9995 0.999 ]

Mean Accuracy:
0.999

Standard Deviation:
0.0005477225575051869
```

In [19]:
```python
# 18. CROSS VALIDATION GRAFİĞİ
# ==========================================================

plt.figure(figsize=(6,4))

plt.hist(cv_scores, bins=5)

plt.title("Cross Validation Accuracy Distribution")

plt.xlabel("Accuracy")

plt.ylabel("Frequency")

plt.show()
```



In [20]:
```
# 19. MODEL KARŞILAŞTIRMA
# ==========================================================

models = ['Logistic Regression', 'Decision Tree', 'Random Forest']

accuracies = [
    lr_accuracy,
    dt_accuracy,
    rf_accuracy
]

plt.figure(figsize=(8,5))

plt.bar(models, accuracies)

plt.title("Model Accuracy Comparison")

plt.ylabel("Accuracy")

plt.show()

print("\n========== FINAL RESULTS ==========")

print("Logistic Regression Accuracy:", lr_accuracy)

print("Decision Tree Accuracy:", dt_accuracy)

print("Random Forest Accuracy:", rf_accuracy)

print("\nProject completed successfully.")
```




```
========== FINAL RESULTS ==========
Logistic Regression Accuracy: 0.9991
Decision Tree Accuracy: 0.9993
Random Forest Accuracy: 0.9993

Project completed successfully.
```

Sonuç

Bu projede kredi kartı dolandırıcılığı tespiti için çeşitli makine öğrenmesi modelleri uygulanmıştır.

Modeller istatistiksel yöntemlerle değerlendirilmiş ve performansları karşılaştırılmıştır.

Sonuçlar:

- Random Forest modelinin en başarılı performansı gösterdiğini,
- veri dengesizliği nedeniyle accuracy metriğinin tek başına yeterli olmadığını,
- confidence interval ve hypothesis testing yöntemlerinin model güvenilirliğini değerlendirmede önemli olduğunu göstermiştir.

Cross Validation sonuçları modelin stabil çalıştığını göstermektedir.